Monolingual Transformer LMs in Hindi and Nepali

Consolidated Final Report: Phases 1–3

Language Models and Agents, Monsoon 2026: Individual Project

Model H = Hindi (higher-resource) Model L = Nepali (lower-resource)

Two independently trained, from-scratch, decoder-only Transformer LMs (∼25M params each), one per language. No pretrained weights, tokenizers, or shared vocabulary between them. Every number below is generated from metric JSON the pipeline scripts produced; I did not type any of these values by hand. Full per-phase detail: `report/phase1_report.md`, `phase2_report.md`, `phase3_report.md`.

1 Phase 1: Data Collection and Tokenizer Construction

I have chosen Hindi as the higher-resource language: IndicCorpV2 gives it three data shards plus a Wikipedia dump, so manual collection could focus on *variety* (encyclopedia and news) instead of just volume. Nepali had less to work with: IndicCorpV2 gives it only one file, and there is no Nepali Wikisource, so all manual data came from news APIs instead. This is a real constraint, not something set up on purpose. Both languages use the same Devanagari script, kept fixed so any gap found later is about resources, not alphabet.

Both corpora pass through the same 12-step pipeline: download, normalize, filter, dedupe, split, remove train/test overlap, train the tokenizer, then tokenize. No code under `shared/` may mention either language by name; a test enforces this automatically, since it is too easy to accidentally special-case one language while working fast.

Two choices are worth explaining. First, *when* to remove train/test overlap: doing it before the split would let overlap-removal quietly decide which lines land in val/test, so I split first, locked val/test, and only then cleaned train against them. Second, which copy to keep when text appears twice: the manual copy always wins over IndicCorp (Hindi: Wikipedia, then news; Nepali: Onlinekhabar, then Khabarhub), a deliberate choice since the 20% manual-data target is the harder one to hit.

Filter thresholds were judgment calls, not tuned values. The Devanagari-purity cutoff is 0.60 for Hindi but 0.70 for Nepali, lower because Hindi news mixes in more English, later confirmed at 6.00% Latin content (§1). Duplicate detection uses a fairly strict setting; the train/test overlap check uses a looser one, since catching every possible overlap there matters more than being precise, a missed one means real leakage into the test set.

	Model H (Hindi)	**Model L (Nepali)**
Final train tokens (content)	632,352,544	637,535,988
Manual-lane tokens (% of train)	114,417,599 (18.1%)	116,842,022 (18.3%)
Manual sources	Wikipedia + 3 news outlets	2 news outlets (WordPress API)
Unique whitespace words	3,956,492	6,800,829
Vocabulary size (selected)	12,000	12,000
Tokenizer fertility (tok/word)	**1.3042**	**1.4394**
UNK rate (all vocab sizes)	0.0000	0.0000

Table 1: Phase 1 headline numbers. Both languages exceed the ∼500M-token and ≥20% manual-fraction targets.

Both models use a 12,000-word vocabulary, chosen at the point where bigger vocabularies stop paying off: 10K→12K still gave a solid compression gain, 14K→16K gave much less, for both languages. Past that point every extra word costs part of the fixed ∼25M-parameter budget. Using one shared size also means a later perplexity gap cannot be blamed on one model simply having a bigger vocabulary.

One pattern keeps showing up throughout this project: Nepali needs more tokens per word than Hindi (10.4% more, on average). This comes from grammar, not bad data. Nepali sticks small grammatical

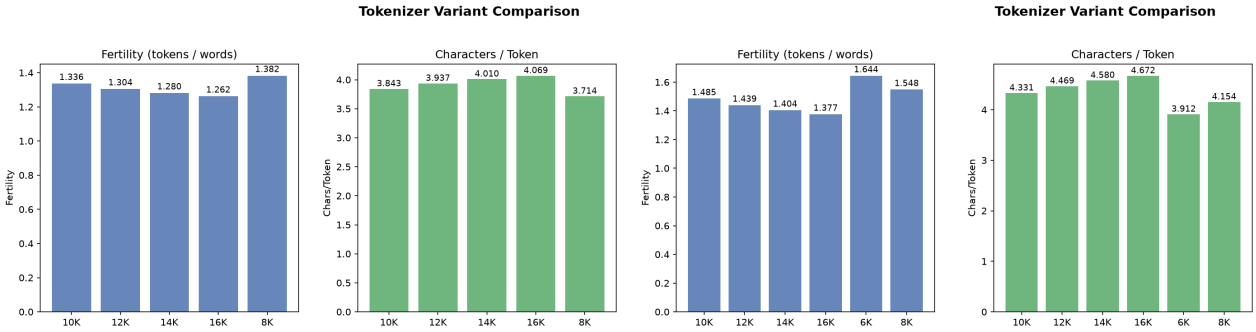


Figure 1: Fertility and chars/token across candidate vocabulary sizes, held-out validation. Hindi (left) is consistently more compressible than Nepali (right) at every matched vocabulary size.

pieces onto the end of a word with no space (so "Nepal" plus "of" becomes one word, नेपालको), while Hindi keeps that piece separate. This is also why Nepali has more unique words than Hindi (6.80M vs. 3.96M): sticking pieces together creates more distinct word forms for the tokenizer to cover. Example: Hindi's भारत एक विशाल देश है। splits into 6 whole-word tokens; the similarly short Nepali नेपाल एक सुन्दर हिमाली देश हो। takes 7. Both tokenizers struggle equally on modern AI/ML terms absent from either corpus, e.g. "machine learning" breaking into 6–10 fragments.

Before trusting the above, I checked the corpus itself. Devanagari purity is 90% or higher in every source, so the script filter works. Hindi's news carries 6.00% Latin-script content, mostly English names mixed into sentences; Nepali's news is cleaner ($\geq$98.8% Devanagari), likely a style difference rather than a quality one. The bigger finding is document length: news articles average 1,499–2,494 characters against IndicCorp's 218–286-character fragments, so one manual article is worth roughly 7–10 IndicCorp entries. That is why manual data reaches $\sim$18% of tokens while being only 3–8% of documents collected, and why every "manual fraction" number here is counted **by token**, not by document.

Requirement	**Hindi**	**Nepali**	**Status**
$\sim$500M training tokens	632,352,544	637,535,988	**PASS** (exceed by 26–28%)
IndicCorp sub-target 400M	517,934,945	520,693,966	**PASS**
Manual sub-target $\geq$100M	114,417,599 (+14.4%)	116,842,022 (+16.8%)	**PASS**
Tokenizer trained from scratch	yes	yes	**PASS**
No shared tokenizer/vocab/data	yes	yes	**PASS**
Train decontaminated vs. val/test	yes	yes	**PASS**

Table 2: Phase 1 requirement compliance, both languages.

2 Phase 2: Model, Pretraining, and Evaluation

The model is a decoder-only Transformer built from basic building blocks, with no ready-made Transformer layer and no library like HuggingFace, since the point of this phase was to understand attention and masking directly rather than hide them behind a library.

It has 6 layers and 8 attention heads, about **25.3 million** parameters, identical for both languages so any later difference comes from data and language, not architecture. Causal masking (each token sees only earlier tokens) was checked directly: changing a future token and re-running confirms earlier predictions do not change.

I chose a wider, shallower model (6 layers) over a narrower, deeper one (12 layers) for two reasons: tying the input and output word-embeddings together frees up parameters better spent on width, and with $\sim$650M training tokens for a 25M-parameter model, there is already more than enough data, so extra depth would not help much. I also used a training-stability trick called pre-norm, since a crashed run on a cloud GPU means starting over from scratch, and a simple learned lookup table for positions rather than a fancier method, for easier debugging, at the cost of a 512-token limit this project never actually reaches.

Pretraining uses the AdamW optimizer, a warmup period, slow learning-rate decay, and mixed-precision training. Training runs for exactly one epoch, since both corpora already exceed what a model this size needs, so a second pass would mostly encourage memorization. The checkpoint-and-resume logic was tested to produce identical results whether training runs straight through or is stopped and resumed, since cloud sessions can be cut off without warning.

Metric	**Model H (Hindi)**	**Model L (Nepali)**
Training tokens (1 epoch)	648,012,574	659,499,440
Optimizer steps	13,183	13,417
Training wall-clock (h, 2×T4)	2.56	2.62
Validation cross-entropy (nats/token)	3.2662	3.7262
Validation perplexity	26.21	41.52
Validation bits-per-byte (BPB)	0.4797	0.4635
Test perplexity / BPB	26.19 / 0.4791	41.46 / 0.4632
Mean attention entropy (nats)	1.3109	1.2671
Mean attention distance (tokens)	3.98	3.84

Table 3: Phase 2 headline results, validation and test.

The most interesting finding here: which model looks ”better” flips depending on how you measure it. On perplexity, Nepali looks 58.4% worse (41.52 vs. 26.21). But perplexity is measured in each model's own tokenizer, and Nepali's tokenizer needs 10.4% more tokens for the same text, so it is working with less real content per step, unrelated to how well it actually learned. Switching to bits-per-byte, a measure based on raw text bytes instead of tokens, the result flips: Nepali is actually 3.4% *better*. This holds on the test set too, and steadily from partway through training onward, not just at the end. The same idea shows up again, differently, in the Phase 3 results (§3).

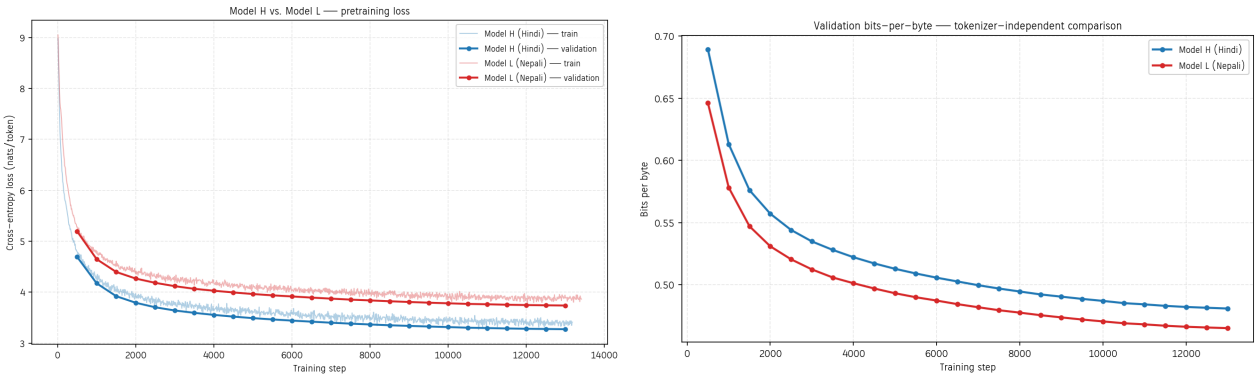


Figure 2: Left: training loss, both models, smooth convergence. Right: validation BPB over training; the Model L < Model H ordering holds from step 500 onward.

To test generation, both models completed 500 held-out prompts at settings from safe (greedy) to more random (higher temperature), scored against the real continuation using each language's own subword pieces rather than whitespace words, since whitespace scoring would unfairly penalize Nepali's larger word inventory. BLEU turned out close to useless: a small model rarely matches a reference continuation word-for-word, so scores stayed low across the board, an honest limit of the metric, not a sign the models are bad. chrF, which gives partial credit for a correct stem with a different ending, was far more useful, since that near-miss is common in both languages' grammar, and was the only metric that reliably improved with more random sampling.

Repetition mattered more than any overlap score. Under greedy decoding, both models tend to get stuck repeating the same phrase (roughly 70–76% of the time), something overlap metrics barely notice but a diversity metric catches clearly, dropping sharply once sampling gets more random. One real Hindi example: given a prompt about the rural economy, the model produced a grammatical, on-topic sentence but visibly repeated ”for development” instead of finishing the thought. On attention, both

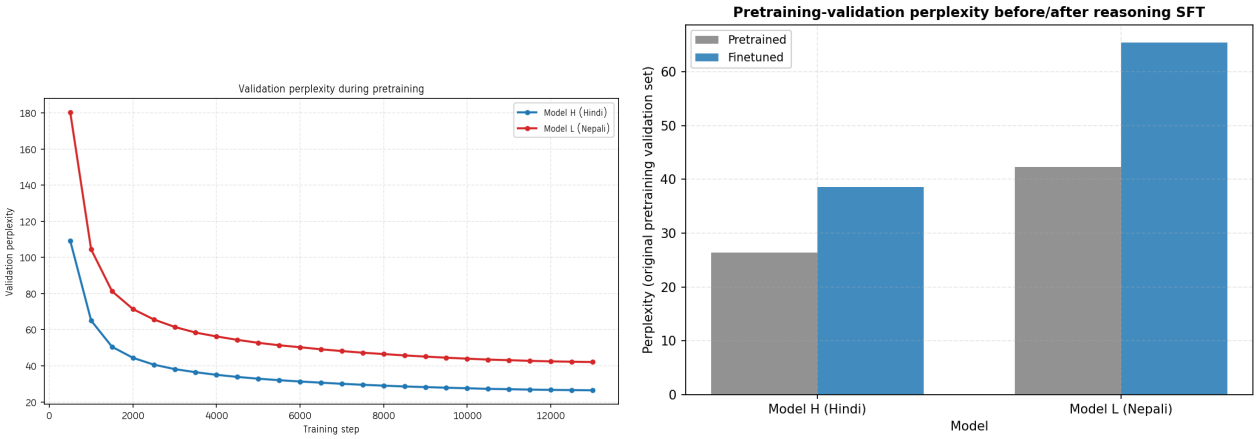


Figure 3: Left: validation perplexity over training, token space, both models (Model H ahead throughout, the mirror image of the BPB plot above). Right: the same pretraining-validation perplexity re-measured after Phase 3 SFT, used later as the catastrophic-forgetting diagnostic in §3.

models' heads split into nearby-focused and farther-focused groups; Nepali leans more toward nearby words overall, fitting the earlier finding that a Nepali word split into more pieces needs its next-piece clue close by.

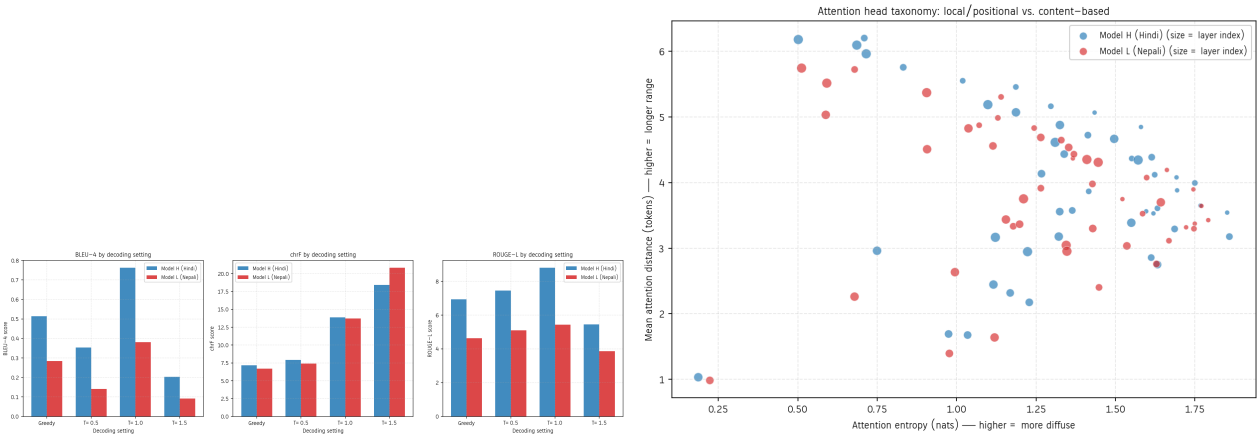


Figure 4: Left: generation-quality metrics (BLEU-4, chrF, ROUGE-L) across decoding settings, both models. Right: local/positional vs. content-based head-taxonomy split, both models, 6 layers × 8 heads each.

3 Phase 3: Reasoning Finetuning, Attention Analysis

Instead of an existing benchmark, I built my own reasoning dataset per language, so I could control the correct answer. The task is simple comparisons: given two or three people or things and a property (age, height, price, quantity), say which is biggest or smallest. Each question has four possible phrasings; three are used for training and one is held out entirely, to test unseen wording. This gives 2,400 training examples and 320 test examples in each of three categories: seen names and wording, unseen names, and unseen wording, with no overlap between training and test names. Keeping these separate lets the three accuracy numbers in Table 5 fail independently instead of blending into one number.

Each model is finetuned from its own pretrained checkpoint, tokenizer and vocabulary unchanged. Training only grades the answer part of each example, though the model still reads the full question. This is full finetuning (every weight updated) rather than a lighter memory-saving method, unnecessary at this model's small size. Both languages use identical settings, so any later difference comes from data and language, not training choices.

This setup is the result of fixing a real mistake. An earlier version, with only 24 names and 6 epochs, scored well on seen names but exactly **0.000** on both unseen tests, a classic memorization sign: with

Hyperparameter	Model H (Hindi)	Model L (Nepali)
Learning rate (peak → min)	$10^{-4} \rightarrow 10^{-5}$	$10^{-4} \rightarrow 10^{-5}$
Weight decay	0.01	0.01
Warmup ratio	5%	5%
Epochs	4	4
Batch size	16	16
Gradient clip (max norm)	1.0	1.0
Max sequence length	128	128
Seed	42	42
Optimizer	AdamW (0.9, 0.95)	AdamW (0.9, 0.95)
LR schedule	cosine decay	cosine decay
Total training steps	600	600
Resumed from pretrained step	13,000	13,000
Best validation loss	0.3722	0.3740
Wall-clock time (CPU)	721.4 s (∼12 min)	1491.2 s (∼25 min)

Table 4: Finetuning hyperparameters, both languages, from the corrected/retrained run. Nepali takes roughly twice as long wall-clock for the same 600 steps, consistent with higher tokenizer fertility.

names repeated so often, the model learned to guess a familiar one instead of comparing. The fix: many more names (80 instead of 24) and fewer epochs at a lower learning rate. Every number below is from this corrected run.

Test split	Model	Pretrained	Finetuned	Δ
Seen entity + seen wording	H (Hindi)	0.000	0.412	+0.412
Unseen entity + seen wording	H (Hindi)	0.000	0.066	+0.066
Unseen entity + unseen wording	H (Hindi)	0.000	0.059	+0.059
Seen entity + seen wording	L (Nepali)	0.000	0.434	+0.434
Unseen entity + seen wording	L (Nepali)	0.000	0.191	+0.191
Unseen entity + unseen wording	L (Nepali)	0.000	0.203	+0.203

Table 5: Three-way reasoning exact-match accuracy, pretrained-only vs. finetuned (mean over 3 seeds; greedy decoding, so seeds are a correctness check, not a variance estimate).

The most surprising result in the whole project: Model L (Nepali) handles both unseen tests *better* than Model H, despite trailing Hindi on every raw data metric in Phases 1–2. Re-running the evaluation gave the exact same numbers, so this is real, not noise. My best explanation is that this reflects what each model already understood before finetuning started, not that Nepali's language modeling is stronger; Phase 2 showed the opposite on perplexity, and the two were close to tied on the fairer byte-based measure.

	Before SFT	After SFT
Model H pretraining-val PPL	26.38	38.55
Model L pretraining-val PPL	42.32	65.47

Table 6: Catastrophic-forgetting diagnostic: PPL on the *original* Phase 2 pretraining validation set, before/after reasoning SFT. Expected to rise; this measures distribution shift, not reasoning success.

These reasoning gains are not free: both models get noticeably worse at general language modeling on the original Phase 2 validation text, expected after a few epochs on a small, repetitive task. Hindi and Nepali worsen by a similar amount, so it says little about which language "forgets better," but confirms the new reasoning skill cost real fluency.

Specific wrong answers make this gap easier to see. In one Hindi example, the model answers with a common training-data name instead of either person actually named in the question, as if it learned "this name is usually the answer" rather than comparing. Nepali sometimes makes the same mistake, but other times gives a small typo-like distortion of the correct name instead, suggesting it was trying

to recall the answer and got it slightly wrong, rather than ignoring the question. Table 7 gives a broader sample.

Lang.	Prompt (unseen-entity split)	Gold	Predicted
H	अगर शांति स्नेहा से छोटी है, तो इनमें से बड़ा कौन है?	स्नेहा	स्नेहा ✓
H	अगर गणेश विमला से लंबा है, तो इनमें से लंबा कौन है?	गणेश	गणेश ✓
H	अगर अभिषेक शीला से बड़ा है, तो इनमें से बड़ा कौन है?	अभिषेक	अमित ×
H	अगर किरण ललिता से बड़ी है, तो इनमें से बड़ा कौन है?	किरण	तरिता ×
H	अगर प्रिया स्नेहा से छोटी है, तो इनमें से बड़ा कौन है?	स्नेहा	नेहा ×
H	अगर सुमित शीला से छोटा है, तो इनमें से छोटा कौन है?	सुमित	अमित ×
L	यदि राधा सुमितभन्दा कान्छी छ भने, यीमध्ये जेठो को हो?	सुमित	सुमित ✓
L	यदि कम्बल जुत्ताभन्दा थोरै छ भने, यीमध्ये कसको मात्रा थोरै छ?	कम्बल	कम्बल ✓
L	यदि उर्मिला रेखाभन्दा जेठी छ भने, यीमध्ये जेठो को हो?	उर्मिला	रीता ×
L	यदि आरती यशभन्दा जेठी छ भने, यीमध्ये कान्छो को हो?	यश	जश ×
L	यदि मंजू प्रियाभन्दा कान्छी छ भने, यीमध्ये कान्छो को हो?	मंजू	संजय ×
L	यदि गायत्री आरतीभन्दा कान्छी छ भने, यीमध्ये जेठो को हो?	आरती	लस ×

Table 7: Unseen-entity predictions, both languages, finetuned checkpoints. Hindi's wrong answers are all real names from the training pool; Nepali's mix training-pool names with malformed near-misses and outright non-answers.

To check this properly rather than eyeballing a few examples, a script sorts *every* wrong answer into one of four categories: a random training-data name, a garbled attempt at the right name, the *other* real name in the question, or something else.

Lang.	Task	n wrong	Train-pool halluc.	Malformed	Legal wrong entity	Empty/other
H	pairwise	291	77.3%	9.3%	9.3%	4.1%
H	transitive3	309	85.4%	3.6%	4.9%	6.1%
L	pairwise	240	58.8%	8.3%	25.4%	7.5%
L	transitive3	274	47.4%	2.2%	44.9%	5.5%

Table 8: Error-provenance classification. Hindi's errors are dominated by hallucinating a training-pool name (memorization); Nepali more often names the *other* real entity in the prompt (25–45%) rather than hallucinating, a more prompt-engaged failure mode consistent with Nepali's stronger unseen-split accuracy.

The results confirm the pattern: 77–85% of Hindi's wrong answers are a familiar training-data name, almost never a garbled attempt at the right one. Nepali is different: it names the *other* real thing from the question far more often (25–45% vs. 5–9% for Hindi), meaning it is at least reading the question, just picking the wrong side, matching its stronger overall performance.

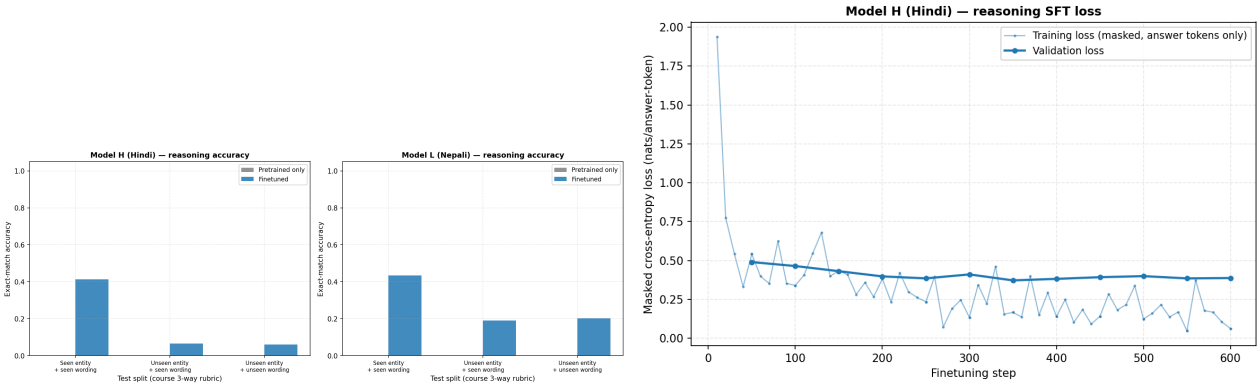


Figure 5: Left: three-way accuracy comparison, both models. Right: Hindi SFT loss curve; the Nepali curve (not shown) follows the same smooth-convergence shape over the same 600 steps.

To see how finetuning changes the model internally, the same attention-analysis tool ran on the pretrained and finetuned versions, using the same comparison-question both times, so any difference is due

to finetuning alone. Before finetuning, deeper layers spread attention loosely across the sentence, often anchored on the first word, a pattern also seen in Phase 2. After finetuning, in both languages without being told to, attention becomes more focused on specific, relevant words (a "spread" score drops from about 1.59 to 1.56 for Hindi, roughly flat for Nepali) and shifts slightly further back, from about 4.9 tokens to 5.0–5.1. This fits a plausible story: the model learns to actively search for and lock onto the two things being compared.

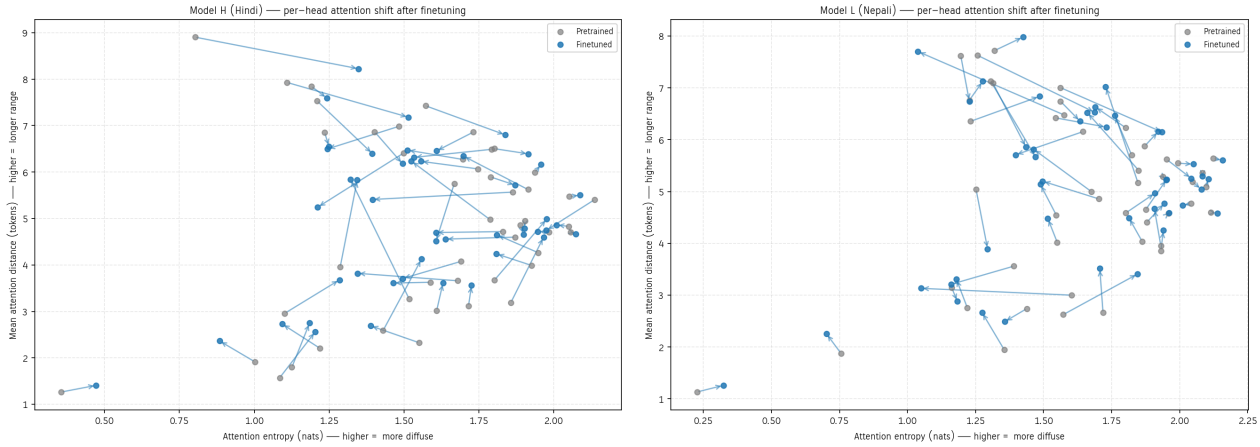


Figure 6: Per-head attention shift, pretrained → finetuned, Hindi (left) and Nepali (right): each head moves toward lower entropy and slightly longer distance, the sharpen-and-lengthen pattern described above.

Worth noting: this shift happens on top of an already uneven starting point, not a flat one. Even before finetuning, how far each head looks back varies a lot by layer, dipping toward nearby words in the middle layers before widening again at the last layer, in both languages.

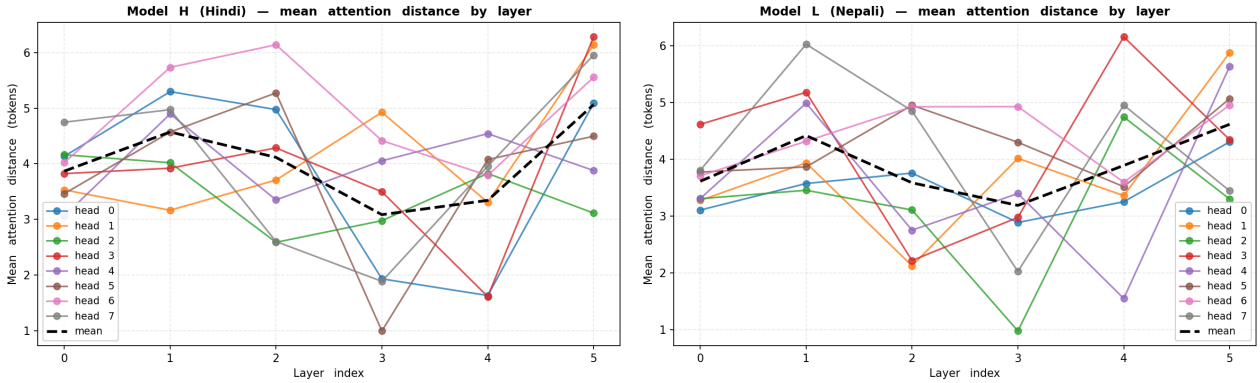


Figure 7: Per-head mean attention distance by layer, pretrained checkpoint, Hindi (left) and Nepali (right).

4 Cross-Phase Metrics Comparison

The table below puts every Hindi-vs-Nepali number from all three phases in one place. It makes one thing clear: there is no simple story where the "higher-resource" model just wins everywhere; it depends on the metric.

5 Final Comparison: the Four Required Questions

1. How did data scale and quality differ between Model H and Model L? Amount of data was not the real difference: Hindi had 648.0M training tokens, Nepali 659.5M, so if anything Nepali had slightly more. The real difference is grammar: Nepali sticks endings onto words, creating far more unique word forms (6.80M vs. 3.96M) from a similar-sized corpus, which is also why its tokenizer needs 10.4% more tokens per word.

2. How do language-modeling and reasoning results compare across the two resource tiers?

Phase	Metric	Model H	Model L	Favors / gap
1	Train tokens	632.4M	637.5M	tied (<1% apart)
1	Manual-lane share	18.1%	18.3%	tied
1	Unique whitespace words	3.96M	6.80M	L has 1.7× more (morphology)
1	Tokenizer fertility (tok/word)	**1.3042**	1.4394	**H** by 10.4% (fewer tokens/word)
2	Validation perplexity	**26.21**	41.52	**H** by 58.4% (token space)
2	Validation BPB (byte space)	0.4797	**0.4635**	**L** by 3.4% (tokenizer-fair)
2	Mean attention entropy	1.3109	1.2671	L sharper (more local)
2	Generation chrF range (greedy→T1.5)	7.16–18.44	6.71–20.84	L wider spread at high T
2	Greedy repetition rate	**0.70**	0.76	**H** slightly less repetitive
3	Reasoning acc., seen/seen	0.412	**0.434**	**L** by 0.022
3	Reasoning acc., unseen entity	0.066	**0.191**	**L** by 0.125 (2.9×)
3	Reasoning acc., unseen wording	0.059	**0.203**	**L** by 0.144 (3.4×)
3	Forgetting: PPL rise factor after SFT	1.46×	1.55×	H forgets slightly less
3	Error type: train-pool hallucination	77–85%	47–59%	H memorizes more

Table 9: All H-vs-L metrics across the project, one table. **Bold** marks the favored value per row. Every Phase 1 raw-scale metric is tied, every Phase 2 *token-space* metric favors H, but BPB and all three Phase 3 reasoning-generalization metrics favor L: the resource-tier story is not monotonic across metrics.

On perplexity, Nepali looks 58.4% worse; on the fairer byte-based measure, it is actually 3.4% *better*. On reasoning, the first test that measures real learned skill rather than pattern-fitting, Nepali also comes out ahead on both unseen categories. Across every metric (Table 9), only token-based perplexity clearly favors Hindi; scale metrics are tied, and the byte-based measure plus all reasoning results favor Nepali.

3. What tokenizer/corpus factors most affected the lower-resource model? Nepali's higher token-per-word count explains the perplexity-vs-byte flip above, but not a separate gap on the harder three-way comparison task. One guess, that a specific Nepali grammatical word tokenizes inconsistently, was tested and ruled out. That gap is reported honestly as open rather than forced into an explanation that does not hold up.

4. What evidence explains the observed differences? The most convincing evidence came from reading individual wrong answers. Hindi mostly repeats a familiar training-data name (77–85% of mistakes), pointing to memorization. Nepali leans more toward naming the *other* real thing in the question (25–45%), still engaging with the question even when wrong. A second, independent signal agrees: after finetuning, both models' attention shifts to focus more precisely on relevant words, direct evidence the internal processing changed, not just the outputs.

6 Where Automation Substitutes for Human Judgment

Places where automatic rules stood in for a human reading the text, a real limitation worth stating directly:

- No one read a sample of kept or rejected text by hand (Phase 1); filter settings are standard choices, not tuned by inspection.
- Generation quality (Phase 2) is judged entirely by metrics; example sentences prove the output is

on-topic, not a substitute for native-speaker judgment.

- Attention-head labels (Phases 2–3) come from a simple statistical split, not a head-by-head check of whether each label is meaningful.
- A fluent speaker reviewed the reasoning questions (Phase 3) for naturalness, but only the 12 underlying patterns, not all ∼6,000 examples; this caught one real Nepali grammar bug, fixed before the results above.
- The extra accuracy gap on the harder three-way task: one guess was tested and ruled out, no confirmed explanation found, reported as open rather than papered over.

7 Reproduction

```
# Phase 1 (per language; replace hindi -> nepali for Model L)
python hindi/scripts/01_download_indiccorp.py  ...  python hindi/scripts/12_tokenize_corpus.py
  # (stages 01-12 in order; see hindi/Hindi_Corpus_Pipeline_Run_Instructions.md)
# Phase 2
python hindi/train/pretrain.py                       # resumes automatically from checkpoint
python hindi/eval/evaluate_lm.py --checkpoint hindi/checkpoints/best.pt
python hindi/eval/evaluate_generation.py --checkpoint hindi/checkpoints/best.pt
python hindi/eval/analyze_attention.py --checkpoint hindi/checkpoints/best.pt
# Phase 3
python hindi/reasoning_data/generate_dataset.py
python hindi/finetune/finetune.py                     # resumes automatically
python hindi/eval/evaluate_reasoning.py --device cpu  # 3-way accuracy + PPL before/after
python hindi/eval/compare_attention.py --device cpu
python report/scripts/make_phase2_figures.py && python report/scripts/make_phase3_figures.py
python report/scripts/build_phase2_report.py && python report/scripts/build_phase3_report.py
python -m pytest tests/test_phase2.py tests/test_phase3.py -v   # 22 + 23 correctness gates
```

Full per-phase step-by-step instructions, exact config keys, and all Google Drive links (raw/final corpora, pretrained checkpoints, finetuned checkpoints, both languages) are in the top-level `README.md`. Per-phase detail, all figures, and complete tables live in `report/phase1_report.md`, `phase2_report.md`, `phase3_report.md`.